

SQL Sample Questions Solutions

1. Customer with the Highest Value Order

Problem Statement

Find the customer who placed the highest-value order and return:

- order_id
 - customer_name
 - total_amount
-

Solution

```
SELECT
  o.order_id,
  c.customer_name,
  o.total_amount
FROM Orders o
JOIN Customers c
  ON o.customer_id = c.customer_id
WHERE o.total_amount = (
  SELECT MAX(total_amount)
  FROM Orders
);
```

Explanation

Step 1

Find the highest order amount.

```
SELECT MAX(total_amount)
```

FROM Orders;

Step 2

Retrieve the order having that amount.

Step 3

Join Customers to get the customer name.

2. Student Enrollments with Grades and Department Names

Problem Statement

Generate a report containing:

- student_id
 - student_name
 - course_id
 - course_name
 - grade
 - department_name
-

Solution

```
SELECT
    s.student_id,
    s.student_name,
    c.course_id,
    c.course_name,
    e.grade,
    d.department_name
FROM Students s
JOIN Enrollments e
    ON s.student_id = e.student_id
```

```
JOIN Courses c
  ON e.course_id = c.course_id
JOIN Departments d
  ON c.department_id = d.department_id
ORDER BY s.student_id, c.course_id;
```

Join Flow

Students
|
Enrollments
|
Courses
|
Departments

Explanation

- Students contains student details.
 - Enrollments contains student-course mappings.
 - Courses contains course information.
 - Departments contains department information.
-

3. Average Assignment Score Per Course

Problem Statement

Calculate the average assignment score for every course.

Return:

- course_name
 - average_score
-

Solution

```
SELECT
  c.course_name,
  ROUND(AVG(s.score), 2) AS average_score
FROM Courses c
JOIN Assignments a
  ON c.course_id = a.course_id
JOIN Submissions s
  ON a.assignment_id = s.assignment_id
GROUP BY
  c.course_id,
  c.course_name
ORDER BY c.course_name;
```

Join Flow

```
Courses
|
Assignments
|
Submissions
```

Explanation

AVG()

Calculates average submission score.

AVG(s.score)

ROUND()

Rounds the result to two decimal places.

ROUND(AVG(s.score), 2)

4. Highest-Revenue Doctor per Department

Problem Statement

For every doctor calculate:

- department_name
- doctor_name
- total_revenue
- revenue_rank

Use:

RANK()

to rank doctors within each department.

Solution

WITH DoctorRevenue AS

```
(
  SELECT
    d.department_name,
    doc.doctor_name,
    SUM(b.bill_amount) AS total_revenue
  FROM Doctors doc
  JOIN Departments d
    ON doc.department_id = d.department_id
  JOIN Appointments a
    ON doc.doctor_id = a.doctor_id
  JOIN Bills b
    ON a.appointment_id = b.appointment_id
  GROUP BY
    d.department_name,
    doc.doctor_name
)
```

SELECT

```
department_name,  
doctor_name,  
total_revenue,  
RANK() OVER (  
    PARTITION BY department_name  
    ORDER BY total_revenue DESC  
) AS revenue_rank  
FROM DoctorRevenue  
ORDER BY department_name, revenue_rank;
```

Execution Flow

Step 1

Calculate revenue per doctor.

```
SUM(b.bill_amount)
```

Step 2

Rank doctors within departments.

```
RANK() OVER (  
PARTITION BY department_name  
ORDER BY total_revenue DESC  
)
```

Example

Cardiology

Doctor A	10000	Rank 1
Doctor B	9000	Rank 2
Doctor C	9000	Rank 2
Doctor D	7000	Rank 4

5. Students Ranked by Course Enrollments

Problem Statement

Assign:

- RANK()
- DENSE_RANK()

based on the number of courses each student is enrolled in.

Solution

WITH StudentCourses AS

```
(
  SELECT
    s.student_id,
    s.name,
    COUNT(e.course_id) AS course_count
  FROM Students s
  LEFT JOIN Enrollments e
    ON s.student_id = e.student_id
  GROUP BY
    s.student_id,
    s.name
)
```

```
SELECT
  student_id,
  name,
  course_count,

  RANK() OVER (
    ORDER BY course_count DESC
  ) AS rank_by_enrollments,

  DENSE_RANK() OVER (
    ORDER BY course_count DESC
  ) AS dense_rank_by_enrollments
```

```
FROM StudentCourses
ORDER BY rank_by_enrollments;
```

Explanation

Count Enrollments

```
COUNT(e.course_id)
```

Counts how many courses a student has enrolled in.

RANK vs DENSE_RANK

Sample Data

Student	Courses
---------	---------

Alice	5
Bob	4
John	4
David	2

RANK()

Alice	1
Bob	2
John	2
David	4

DENSE_RANK()

Alice	1
Bob	2
John	2
David	3

6. Movies Watched by Viewers of 'Life of Pi'

Problem Statement

Find all movies watched by users who have watched:

Life of Pi

Return:

- movie_id
- title

Duplicates should not appear.

Solution

```
SELECT DISTINCT
  m.movie_id,
  m.title
FROM Movies m
JOIN ViewingHistory vh
  ON m.movie_id = vh.movie_id
WHERE vh.user_id IN
(
  SELECT DISTINCT vh2.user_id
  FROM ViewingHistory vh2
  JOIN Movies m2
    ON vh2.movie_id = m2.movie_id
  WHERE m2.title = 'Life of Pi'
)
ORDER BY m.title;
```

Alternative Using EXISTS

```
SELECT DISTINCT
```

```
    m.movie_id,  
    m.title  
FROM Movies m  
JOIN ViewingHistory vh  
    ON m.movie_id = vh.movie_id  
WHERE EXISTS  
(  
    SELECT 1  
    FROM ViewingHistory vh2  
    JOIN Movies m2  
        ON vh2.movie_id = m2.movie_id  
    WHERE m2.title = 'Life of Pi'  
        AND vh2.user_id = vh.user_id  
);
```

Execution Flow

Step 1

Find all users who watched:

'Life of Pi'

Step 2

Find every movie watched by those users.

Step 3

Remove duplicates using:

DISTINCT

SQL Concepts Covered

Problem

Concepts

Highest Value Order	MAX(), Subquery
Student Enrollments	INNER JOIN
Average Assignment Score	AVG(), GROUP BY
Highest Revenue Doctor	SUM(), CTE, RANK()
Student Enrollment Ranking	COUNT(), RANK(), DENSE_RANK()
Movies Watched by Viewers of Life of Pi	DISTINCT, IN, EXISTS
All Problems	Joins, Aggregations, Subqueries, Window Functions, CTEs

Interview Questions Based on These Problems

Q1. Difference between RANK() and DENSE_RANK()?

- RANK() skips ranks after ties.
 - DENSE_RANK() does not skip ranks.
-

Q2. Why use a CTE?

- Improves readability.
 - Breaks complex queries into logical steps.
-

Q3. Difference between EXISTS and IN?

EXISTS	IN
Stops after first match	Evaluates full list
Better for large datasets	Simpler syntax

Q4. Why use DISTINCT?

To remove duplicate rows from the result set.

Q5. Why use LEFT JOIN instead of INNER JOIN in student ranking?

To include students who are not enrolled in any courses.

Student with no enrollments → Course Count = 0
